CS370 Term Project - Milestone 2 Design Document

Abdulrahman Alenazi and Nailah Lewis

1. Project Overview

For our term project, we decided to build a fridge door and temperature monitor using a Raspberry Pi. I liked this idea because sometimes I forget to close the fridge door completely, especially when I am in a hurry. If the door stays open, the temperature can increase, food can spoil, and electricity can be wasted.

Our device will use two sensors. The first is a magnetic reed switch that detects whether the fridge door is open or closed. The second is a temperature sensor that measures the temperature inside the fridge.

I want our program to use both readings together. For example, opening the fridge for a few seconds is normal, but leaving it open for several minutes while the temperature increases could be a problem. We also want to check how long the fridge takes to cool down after the door closes.

The main goal is to make something useful that can monitor the fridge automatically and save information for us to review later.

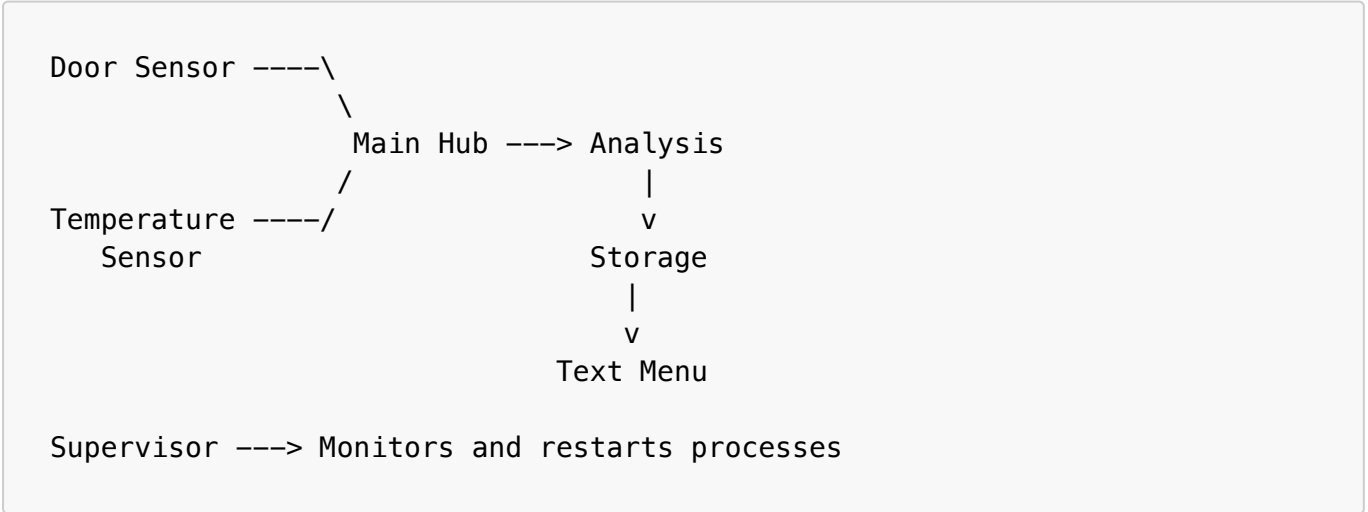
2. System Architecture

We are using a Raspberry Pi and writing our programs in C. Our plan is to use separate processes for the two sensors so they can work independently.

The door sensor will check its GPIO input every 50 milliseconds. The temperature sensor will take a reading about every 2 seconds, depending on the sensor model.

Both processes will send their readings to a main hub using Unix domain sockets. We chose sockets because all the processes will run on the same Raspberry Pi.

Architecture Diagram



The main hub will collect readings and organize them using timestamps. The analysis process will look for things like long door openings and unusual temperature changes.

We also plan to make a simple text menu that lets us check the current readings, recent events, and saved information.

3. Operating Systems Mechanisms

For this project, we chose mechanisms D and E from Section 3.2.

Mechanism D - Custom Storage Layer

I want the device to save readings even when nobody is watching it. We plan to build a small append-only storage system in C.

Each record will include a timestamp, sensor type, and reading or event. We will also include record lengths and checksums to help detect damaged or incomplete records.

Instead of writing every reading separately, we plan to use a buffer and write groups of records. For important events, we will use `fsync()` to make sure the information is saved to the SD card.

If the Raspberry Pi loses power, the program will check the log during startup. It will keep complete, valid records and remove any incomplete record at the end.

This is important because I do not want a power interruption to erase everything the device recorded.

Mechanism E - Multi-Process Architecture

Our second mechanism is using multiple processes. We chose this because I do not want one sensor problem to stop the entire device.

The door sensor and temperature sensor will run separately. A hub will receive their readings through Unix domain sockets.

We also plan to create a supervisor that checks whether the processes are still running. If a process crashes, the supervisor will detect the failure, record it, and try to restart the process.

For example, if the temperature process stops, the door sensor should continue working. The system should show that the temperature reading is unavailable instead of displaying old information as if it were current.

This design should make the device more reliable during long periods of use.

4. Failure Modes and Recovery

We know that problems can happen while the device is running, so we made a plan for detecting and handling them.

Problem	**Detection**	**Recovery**
Door sensor issue	Incorrect response during testing	Check wiring and report uncertain status
Temperature sensor failure	Invalid or missing readings	Retry and mark reading unavailable

Problem	Detection	Recovery
Sensor process crashes	Supervisor detects process exit	Log failure and restart
Hub stops receiving data	Missing updates or timeout	Reconnect or restart
Storage becomes full	Write operation fails	Report error and stop claiming data was saved
Power interruption	Startup log validation	Recover valid records

One limitation of the magnetic reed switch is that a disconnected wire might look like an open door. We cannot always identify that problem from the signal alone.

We will test the sensor by moving the magnet toward and away from the switch and comparing the output with its actual position.

We will also stop a sensor process on purpose to check whether the supervisor can restart it.

5. Storage Design

Our storage system will keep information about the fridge throughout the day.

We plan to save:

- Date and time
- Sensor name and reading
- Door open or closed
- Temperature in Celsius
- Door-open duration
- Important events
- Process failures and restarts

The door sensor will check every 50 milliseconds, but we will mainly save records when the door status changes. This avoids storing the same information repeatedly.

The temperature sensor will save readings about every 2 seconds.

We plan to use an append-only binary log with record lengths and checksums. During startup, the program will check each record to make sure it is valid.

Our initial goal is to keep at least seven days of data, depending on available storage. Older records can be archived or removed when necessary.

6. Hardware Limitations

One challenge is that we are using inexpensive sensors, so their readings will not always be perfect.

The magnetic reed switch only detects whether the magnet is close enough to activate it. It cannot measure exactly how far the fridge door is open.

The temperature sensor might also take some time to respond when the door opens. Because of this, we cannot assume every temperature increase means something is wrong.

Instead, we will focus on detecting long door openings and unusually slow cooling.

Another limitation is that Nailah currently has the Raspberry Pi and both sensors. I am writing the door sensor code using VS Code on my Mac, while Nailah will connect the hardware and test the programs on the Raspberry Pi.

We will need to confirm the GPIO connections and required libraries before testing everything together.

If we use recorded or simulated data for testing, we will clearly identify it instead of presenting it as live sensor output.

7. Evaluation Plan

We want to test whether our device actually works instead of only checking whether the programs run.

Test	Our Goal
Open and close the door 20 times	At least 95% correct detection
Door response time	Less than 250 milliseconds
Temperature comparison	Within 1.5 degrees Celsius
Stop a sensor process	Restart within 5 seconds
Interrupt a storage write	Recover valid records
CPU usage	Under 10% average total
Memory usage	No continuous increase
Continuous operation	48 hours without unrecovered failures

For the door sensor, we will move the magnet close to and away from the switch 20 times and compare the results.

For temperature, we will compare the sensor with another thermometer if one is available.

We will also test longer door openings and observe how the temperature changes after closing the door.

During the 48-hour test, we plan to record hourly updates, process memory usage, and errors. These are our testing goals, not results we have already achieved.

8. Team Responsibilities

My Part - Abdulrahman

I am responsible for writing the Milestone 2 design document and developing the magnetic door sensor program in C.

I am using VS Code to write the program. It reads the GPIO input, checks whether the door is open or closed, and prints the status with a timestamp.

I also created a Makefile and instructions so Nailah can compile and test my code on the Raspberry Pi.

After the door sensor is working, I will help connect it to the main system and test it with the temperature sensor.

Nailah's Part

Nailah has the Raspberry Pi and both sensors. She is responsible for connecting the hardware and developing the temperature sensor code.

She will also test my door sensor code on the Raspberry Pi and record a video showing the sensors working.

We will work together on the main hub, storage, process supervision, and final testing.

9. Development Tools and AI Usage

We are using VS Code, GitHub, and C for our project. I am using ChatGPT to help organize the design document and work through the door sensor setup.

We also plan to use Claude Code to review code, understand Linux system calls, and help debug problems. We will keep the required original Claude Code transcripts and document important development decisions in `PROMPTLOG.md`.

Our actual Raspberry Pi device will not depend on ChatGPT, Claude Code, or another online AI service. The monitoring and storage programs will run locally.

10. Next Steps

Our next step is to get both sensors working on the Raspberry Pi and collect real readings.

Once the door sensor and temperature sensor are working, we will connect them to the main hub and storage system.

For Milestone 3, we need both sensors sending real readings through the complete pipeline into storage, with at least one menu mechanism working.

We will continue testing the device and update our design if we find problems.